Rapport Technique

CUDA Ray Tracer

Moteur de rendu physiquement réaliste

GPU (CUDA) / CPU (OpenMP) / Mode interactif (OpenGL)

Projet RayTracing

Mars 2026

Résumé

Ce rapport présente l'implémentation d'un moteur de ray tracing physiquement réaliste tirant parti des capacités de calcul parallèle des GPU modernes via CUDA. Le projet propose deux chemins de rendu : un chemin GPU utilisant CUDA pour des performances optimales (160 MRays/s sur RTX 4060), et un chemin CPU multi-thread utilisant OpenMP. Un mode interactif en temps réel est également disponible grâce à l'interopérabilité CUDA-OpenGL. L'accélération GPU est de l'ordre de $\times$**56** par rapport à l'exécution CPU.

Table des matières

1 Introduction **3**
1.1 Objectifs . . . 3
1.2 Technologies utilisées . . . 3

2 Architecture générale **3**
2.1 Organisation des modules . . . 4
2.2 Trois modes de fonctionnement . . . 4

3 Algorithme de path tracing **4**
3.1 Principe du Monte Carlo path tracing . . . 4
3.2 Traversée BVH itérative . . . 4
3.3 Accumulation et tone mapping . . . 5

4 Matériaux **5**
4.1 Lambertian (diffus) . . . 5
4.2 Métal (réflexion spéculaire) . . . 5

4.3 Diélectrique (verre/réfraction) 5

5 Géométrie **6**
5.1 Sphère 6
5.2 Plan 6

6 Caméra **6**

7 Implémentation GPU (CUDA) **6**
7.1 Organisation des threads 6
7.2 Kernels CUDA 7
7.3 Gestion mémoire 7
7.4 Initialisation RNG rapide 7

8 Implémentation CPU (OpenMP) **7**
8.1 Parallélisation 7
8.2 Générateur aléatoire CPU 8
8.3 Comparaison GPU vs CPU 8

9 Mode interactif OpenGL **8**
9.1 Architecture CUDA-OpenGL Interop 8
9.2 Contrôles et accumulation progressive 8

10 Tests unitaires **8**

11 Optimisations de performance **9**

12 Analyse comparative CPU vs GPU **9**
12.1 Intérêts de la solution GPU (CUDA) 9
12.2 Intérêts de la solution CPU (OpenMP) 10
12.3 Discussion du speedup effectif 10

13 Conclusion **10**

1 Introduction

Le ray tracing est une technique de rendu qui simule le comportement physique de la lumière en traçant des rayons depuis la caméra vers la scène. Contrairement à la rastérisation, cette approche permet de produire naturellement des effets tels que les réflexions, les réfractions, les ombres douces et la profondeur de champ.

Ce projet implémente un **path tracer Monte Carlo** : pour chaque pixel, de nombreux rayons sont lancés avec des perturbations aléatoires, et leurs contributions sont moyennées pour obtenir une image convergée. La nature massivement parallèle du problème (chaque pixel est indépendant) le rend idéal pour les GPU.

1.1 Objectifs

- Implémenter un path tracer complet avec matériaux physiques (Lambertian, Métal, Diélectrique)
- Exploiter CUDA pour une accélération GPU massive
- Fournir un chemin de rendu CPU avec OpenMP comme alternative portable
- Développer un mode interactif temps réel via l'interopérabilité CUDA-OpenGL
- Construire un pipeline de tests unitaires sans dépendance CUDA

Le code source complet est disponible sur GitHub : `https://github.com/Gotman08/RayTracing.git`

Le système de build utilise CMake 3.18+ avec trois modes : build GPU complet (défaut), build CPU seul (`BUILD_CPU_ONLY=ON`), et tests seuls (`BUILD_TESTS_ONLY=ON`). Les images sont exportées aux formats PNG, BMP, JPG, TGA et PPM.

1.2 Technologies utilisées

Technologie	Rôle
CUDA 12.6	Rendu GPU parallèle
OpenMP	Rendu CPU multi-thread
GLFW 3.3+	Fenêtrage (mode interactif)
OpenGL 3.3+	Affichage temps réel
stb_image_write	Export PNG/BMP/JPG/PPM
CMake 3.18+	Système de build
CTest	Tests unitaires

Table 1 – Technologies et bibliothèques du projet

2 Architecture générale

Le projet est structuré autour de bibliothèques d'en-têtes (fichiers `.cuh`) organisées par domaine fonctionnel, ce qui favorise la réutilisation du code entre les chemins GPU et CPU.

2.1 Organisation des modules

Module	Contenu
Utilitaires core	`vec3`, `ray`, `aabb`, `interval`, `timer`, générateur aléatoire
Géométrie	`sphere`, `plane`, interface `hittable`
Matériaux	`lambertian`, `metal`, `dielectric`
Accélération	Arbre BVH (variantes GPU et CPU)
Rendu	Renderers GPU/CPU, tone mapping, dispatch de matériaux
Caméra	Caméra perspective avec DoF et motion blur
Mode interactif	Gestionnaire de fenêtre, interop GL, contrôleur caméra, gestion des entrées

Table 2 – Organisation modulaire du projet

2.2 Trois modes de fonctionnement

1. **Mode GPU** (défaut) : rendu CUDA, export image
2. **Mode CPU** (`-cpu`) : rendu OpenMP, export image
3. **Mode interactif** (`-interactive`) : rendu CUDA temps réel avec affichage OpenGL

3 Algorithme de path tracing

3.1 Principe du Monte Carlo path tracing

Pour chaque pixel (i, j), on lance N rayons avec des perturbations aléatoires. La couleur finale est la moyenne des contributions :

$$C(i, j) = \frac{1}{N} \sum_{k=1}^{N} \text{trace}(r_k)$$

Chaque rayon est suivi de rebond en rebond jusqu'à une profondeur maximale $d_{\max}$ ou jusqu'à ce qu'il s'échappe vers l'arrière-plan.

3.2 Traversée BVH itérative

L'accélération par **Bounding Volume Hierarchy** (BVH) réduit la complexité de l'intersection de $O(n)$ à $O(\log n)$. La traversée est implémentée de façon itérative avec une pile (profondeur max : 64 nœuds), ce qui est critique pour les performances GPU où la récursion est coûteuse.

La construction du BVH utilise une stratégie de *median split* sur l'axe le plus long, avec partitionnement par centroïde.

3.3 Accumulation et tone mapping

Les couleurs sont accumulées en virgule flottante (HDR). La compression vers l'espace SDR utilise le **tone mapping de Reinhard** :

$$c' = \frac{c}{1+c}$$

Suivi d'une **correction gamma** $\gamma = 2.2$:

$$c_{\text{out}} = c'^{1/\gamma}$$

4 Matériaux

4.1 Lambertian (diffus)

Le matériau Lambertien modélise les surfaces mattes avec une distribution cosinus-pondérée :

$$\boldsymbol{d}_{\text{scatter}} = \hat{\boldsymbol{n}} + \boldsymbol{r}_{\text{unit}}$$

où $\boldsymbol{r}_{\text{unit}}$ est un vecteur unitaire aléatoire. L'atténuation est la couleur *albedo* du matériau.

4.2 Métal (réflexion spéculaire)

La réflexion spéculaire avec flou contrôlé :

$$\boldsymbol{d}_{\text{reflect}} = \boldsymbol{v} - 2(\boldsymbol{v} \cdot \hat{\boldsymbol{n}})\hat{\boldsymbol{n}} + f \cdot \boldsymbol{r}_{\text{unit}}$$

où $f \in [0, 1]$ est le paramètre de *fuzz* contrôlant le flou des réflexions ($0 =$ miroir parfait).

4.3 Diélectrique (verre/réfraction)

Le diélectrique combine réflexion et réfraction via **l'approximation de Schlick** pour les équations de Fresnel :

$$R(\theta) \approx R_0 + (1 - R_0)(1 - \cos\theta)^5, \quad R_0 = \left(\frac{n_1 - n_2}{n_1 + n_2}\right)^2$$

La réfraction suit la loi de Snell-Descartes :

$$n_1 \sin\theta_1 = n_2 \sin\theta_2$$

La **réflexion interne totale** est gérée : si $n_1/n_2 \cdot \sin\theta_1 > 1$, le rayon est entièrement réfléchi. L'indice de réfraction par défaut est $n = 1.5$ (verre).

Matériau	Modèle	Paramètres	Usage
Lambertian	Diffus cosinus	albedo (RGB)	Surfaces mattes
Metal	Réflexion + flou	albedo (RGB), fuzz [0,1]	Métaux, miroirs
Dielectric	Snell + Schlick	IOR (défaut 1.5)	Verre, eau

Table 3 – Résumé des matériaux implémentés

5 Géométrie

5.1 Sphère

L'intersection rayon-sphère est résolue analytiquement via l'équation quadratique :

$$\|\boldsymbol{o} + t\boldsymbol{d} - \boldsymbol{c}\|^2 = r^2$$

soit $at^2 + bt + c = 0$ avec $a = \|\boldsymbol{d}\|^2$, $b = 2\boldsymbol{d} \cdot (\boldsymbol{o} - \boldsymbol{c})$, $c = \|\boldsymbol{o} - \boldsymbol{c}\|^2 - r^2$.

La normale de surface est calculée correctement pour les rayons entrant et sortant (front/back face). Les coordonnées UV sphériques sont également calculées pour un potentiel texturage.

La AABB de la sphère est le cube $[\boldsymbol{c} - r, \boldsymbol{c} + r]$ sur chaque axe.

5.2 Plan

Le plan infini est supporté comme primitive géométrique supplémentaire, permettant la construction de sols et de surfaces plates.

6 Caméra

La caméra utilise une projection perspective avec FOV vertical configurable (défaut : 20°), position (*lookfrom*), cible (*lookat*) et vecteur up.

La **profondeur de champ** est simulée via un modèle de lentille mince : les rayons sont échantillonnés depuis un disque d'ouverture, pointant vers le plan de mise au point (défaut : $f/22$, distance 10 unités). Le **motion blur** est obtenu en générant les rayons à des instants aléatoires entre les temps d'ouverture et de fermeture de l'obturateur.

7 Implémentation GPU (CUDA)

7.1 Organisation des threads

Le rendu est organisé en grille 2D de blocs 16×16 (256 threads/bloc) :

$$\text{grid} = \left(\left\lceil \frac{W}{16} \right\rceil, \left\lceil \frac{H}{16} \right\rceil \right)$$

Chaque thread traite un pixel de l'image de manière indépendante.

7.2 Kernels CUDA

Kernel	Rôle
`init_rand_states_fast`	Initialisation des états RNG (Wang hash)
`render_kernel`	Rendu principal (SPP samples par pixel)
`render_kernel_accumulate`	Accumulation progressive (mode interactif)
`convert_to_rgba8`	Conversion HDR→RGBA8 avec tone mapping

Table 4 – Kernels CUDA du projet

7.3 Gestion mémoire

- **Host→Device** : matériaux, nœuds BVH, primitives (via `cudaMemcpy`)
- **Device→Host** : framebuffer final après rendu
- **Fixup de pointeurs** : les pointeurs de matériaux host sont redirigés vers les adresses device en un seul passage
- **États RNG** : un `curandState` par pixel (mémoire device)

7.4 Initialisation RNG rapide

Au lieu de `curand_init` (coûteux), le projet utilise un **Wang hash** pour une initialisation O(1) par pixel :

```
__device__ uint32_t wang_hash(uint32_t seed) {
    seed = (seed ^ 61) ^ (seed >> 16);
    seed *= 9;
    seed = seed ^ (seed >> 4);
    seed *= 0x27d4eb2d;
    seed = seed ^ (seed >> 15);
    return seed;
}
```

Listing 1 – Wang hash pour l'initialisation RNG

8 Implémentation CPU (OpenMP)

8.1 Parallélisation

Le rendu CPU utilise OpenMP avec un ordonnancement dynamique par lignes :

```
#pragma omp parallel for schedule(dynamic, 1)
for (int j = 0; j < height; j++) {
    CPURandom rng(seed + j);
    for (int i = 0; i < width; i++) {
        // rendu du pixel (i, j)
    }
}
```

Listing 2 – Parallélisation OpenMP

L'**ordonnancement dynamique** (`schedule(dynamic, 1)`) assure un équilibrage de charge optimal, certaines lignes étant plus coûteuses que d'autres selon la complexité de la scène.

8.2 Générateur aléatoire CPU

Un générateur à congruence linéaire (`CPURandom`) est utilisé, initialisé par ligne pour garantir la reproductibilité et éviter les conditions de course entre threads.

8.3 Comparaison GPU vs CPU

Paramètre	**GPU (RTX 4060)**	**CPU (8 threads)**
Résolution	800×600	800×600
Samples per pixel	100	100
Temps de rendu	∼0.3 s	∼17 s
Débit	∼160 MRays/s	∼2.8 MRays/s
Speedup GPU/CPU	**×56**	

Table 5 – Comparaison des performances GPU vs CPU

9 Mode interactif OpenGL

9.1 Architecture CUDA-OpenGL Interop

Le mode interactif exploite l'interopérabilité CUDA-OpenGL pour afficher les frames rendues par CUDA sans aller-retour par le CPU :

1. **PBO** (Pixel Buffer Object) OpenGL alloué en mémoire GPU
2. Enregistrement du PBO comme ressource CUDA (`cudaGraphicsResource`)
3. À chaque frame : `map_for_cuda()` → kernel `convert_to_rgba8` → `unmap_from_cuda()` → `glTexSubImage2D` → rendu quad plein écran

Ce pipeline **zéro copie CPU** maximise les performances temps réel.

9.2 Contrôles et accumulation progressive

Le contrôleur caméra offre un déplacement WASD avec orientation souris, et des raccourcis clavier (R : reset, P : capture, Échap : quitter). Le nombre de samples par frame est configurable : 1 en mouvement (réactivité) et jusqu'à 1000 à l'arrêt (qualité progressive).

10 Tests unitaires

La suite de tests est compilable **sans CUDA** (`BUILD_TESTS_ONLY=ON`), ce qui permet de valider les composants mathématiques et géométriques de manière indépendante du GPU.

Fichier	Composant testé
`test_vec3.cpp`	Opérations vectorielles (construction, arithmétique, normalisation, produits
`test_ray.cpp`	Paramétrisation des rayons
`test_interval.cpp`	Intersection et containment d'intervalles
`test_aabb.cpp`	Intersection et fusion de boîtes englobantes
`test_sphere.cpp`	Intersection rayon-sphère
`test_plane.cpp`	Géométrie des plans
`test_materials.cpp`	Matériaux (Lambertian, Métal, Diélectrique)
`test_camera.cpp`	Génération de rayons, FOV, profondeur de champ
`test_tone_mapping.cpp`	Tone mapping Reinhard et correction gamma
`test_bvh.cpp`	Construction et traversée du BVH
`test_main.cpp`	Framework de tests et statistiques

Table 6 – Suite de tests unitaires

La compilation et l'exécution se font via `cmake .. -DBUILD_TESTS_ONLY=ON && make && ctest`. L'ensemble de la suite comprend **144 tests** validant toutes les classes du projet.

11 Optimisations de performance

1. **Wang hash RNG** : initialisation O(1) des états aléatoires par pixel, évitant le coûteux `curand_init`
2. **Traversée BVH itérative** : pile explicite (64 nœuds max) évitant la récursion, favorable au cache GPU et au parallélisme SIMT
3. **Fast math** : flags `--use_fast_math` (GPU) et `-ffast-math` (CPU) pour des opérations FPU approximatives mais plus rapides
4. **Intersection AABB sans branchement** : code déroulé pour minimiser les divergences de warp sur GPU
5. **Ordonnancement dynamique CPU** : `schedule(dynamic, 1)` pour un équilibrage de charge optimal entre threads OpenMP

12 Analyse comparative CPU vs GPU

12.1 Intérêts de la solution GPU (CUDA)

Le GPU est idéal pour le path tracing grâce à son architecture massivement parallèle : des milliers de cœurs CUDA traitent chaque pixel de manière indépendante. La RTX 4060 (3072 cœurs CUDA) atteint **160 MRays/s**, soit un débit 56 fois supérieur au CPU.

Avantages : débit très élevé, adapté aux problèmes embarrassingly parallel, mémoire GPU à haute bande passante (GDDR6).

Inconvénients : transferts mémoire Host↔Device coûteux, gestion mémoire complexe (fixup de pointeurs, allocation explicite), débogage plus difficile, portabilité limitée aux GPU NVIDIA.

12.2 Intérêts de la solution CPU (OpenMP)

Le CPU offre portabilité et simplicité. OpenMP permet une parallélisation à faible coût de développement (une seule directive pragma), et le débogage se fait avec des outils standards (GDB, Valgrind).

Avantages : portabilité universelle, mémoire partagée (pas de transferts), débogage direct, support automatique de tous les compilateurs C++ modernes.

Inconvénients : parallélisme limité à 8–16 threads (vs milliers sur GPU), débit de seulement **2.8 MRays/s** sur 8 threads.

12.3 Discussion du speedup effectif

Le speedup mesuré de $\times$**56** s'explique par la nature embarrasingly parallel du path tracing : chaque pixel étant totalement indépendant, le GPU exploite pleinement ses milliers de cœurs sans synchronisation. Ce ratio est cohérent avec la littérature sur les accélérations GPU pour les applications de rendu Monte Carlo.

Le mode interactif illustre une approche **hybride** : CUDA effectue le rendu, tandis qu'OpenGL (via l'interop PBO zéro copie) gère l'affichage. Cette combinaison permet un rendu temps réel avec accumulation progressive, démontrant la complémentarité des deux technologies.

13 Conclusion

Ce projet démontre une implémentation complète et optimisée d'un path tracer physiquement réaliste exploitant le parallélisme massif des GPU modernes.

Les points forts du projet sont :

- **Double implémentation GPU/CPU** offrant flexibilité et portabilité
- **Accélération $\times$56** grâce à CUDA par rapport au rendu CPU
- **Mode interactif temps réel** via l'interop CUDA-OpenGL zéro copie
- **Matériaux physiques** couvrant les principaux types de surfaces réelles
- **Pipeline de tests** indépendant de CUDA pour une validation fiable
- **Build système flexible** avec CMake supportant plusieurs configurations
- **Architecture modulaire** facilitant l'extension avec de nouveaux matériaux ou primitives géométriques

Les perspectives d'amélioration incluent l'ajout de sources de lumière explicites, le support des textures, l'implémentation du sampling par importance multiple (MIS), et l'extension aux volumes participatifs (brouillard, fumée).